EXPERIMENT 5

D15A-57

Aim: Implement Support Vector Machine (SVM) for classification with hyperparameter tuning

Theory:

1. Dataset Source
Custom dataset : Link

2. Dataset Description

The dataset contains time-series fitness activity records collected from wearable fitness trackers.

Features used in this experiment:

1. timestamp – Date and time at which the activity was recorded
2. steps – Number of steps recorded

3. Mathematical Formulation of the Algorithm

Internally, SVM treats classes as:

$y_i \in \{-1,+1\}$

Soft margin SVM

$$\min_{w,b,\xi} \frac{1}{2}||w||^2 + C\sum_{i=1}^{n} \xi_i$$

Subject to:

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i$$

w = weight vector

b = bias

C= regularization parameter

Decision function

$f(x)=w \cdot x+b$

4. Algorithm Limitations

- Linear assumption
  Since kernel='linear', the algorithm assumes that classes are linearly separable in the transformed feature space. If the true boundary is nonlinear, performance may decrease

- Sensitivity to scaling
  SVM is sensitive to feature magnitude. Without StandardScaler, the feature with larger numeric range would dominate.

- Not ideal for very large datasets
  Training time increases significantly with large datasets.

- Limited interpretability
  In higher dimensions, understanding model decisions becomes difficult.

- Median-based labeling limitation
  The label is artificially created using median threshold. This may not reflect real-world activity classification.

5. Methodology / Workflow

Step 1: Load all CSV files using glob and concatenate into a single dataframe.

Step 2: Clean column names and convert:

- timestamp → datetime

- steps → numeric

Step 3: Perform hourly aggregation:

- Compute total_steps, mean_steps, std_steps.

Step 4: Handle missing values using fillna(0).

Step 5: Create binary labels using median threshold of total_steps.

Step 6: Select features:

- total_steps

- std_steps

Step 7: Scale features using StandardScaler.

Step 8: Split dataset:

- 70% training
- 30% testing
- Stratified split to maintain class balance

Step 9: Train SVM:

- Kernel: linear
- class_weight='balanced'

Step 10: Predict on test data.

Step 11: Evaluate using:

- Accuracy
- Confusion matrix

Step 12: Plot decision boundary for visualization.

6. Performance Analysis

The model is evaluated using **accuracy** and a **confusion matrix**.

Accuracy measures the proportion of correctly classified samples in the test set.

The confusion matrix shows how many instances were correctly and incorrectly classified as Low Activity and High Activity.

Since the dataset was balanced using median-based labeling and stratified splitting, accuracy is an appropriate performance metric for this experiment.

7. Hyperparameter Tuning

C (regularization parameter)

Smaller C increases margin width but allows more misclassification.
Larger C reduces misclassification but may overfit.

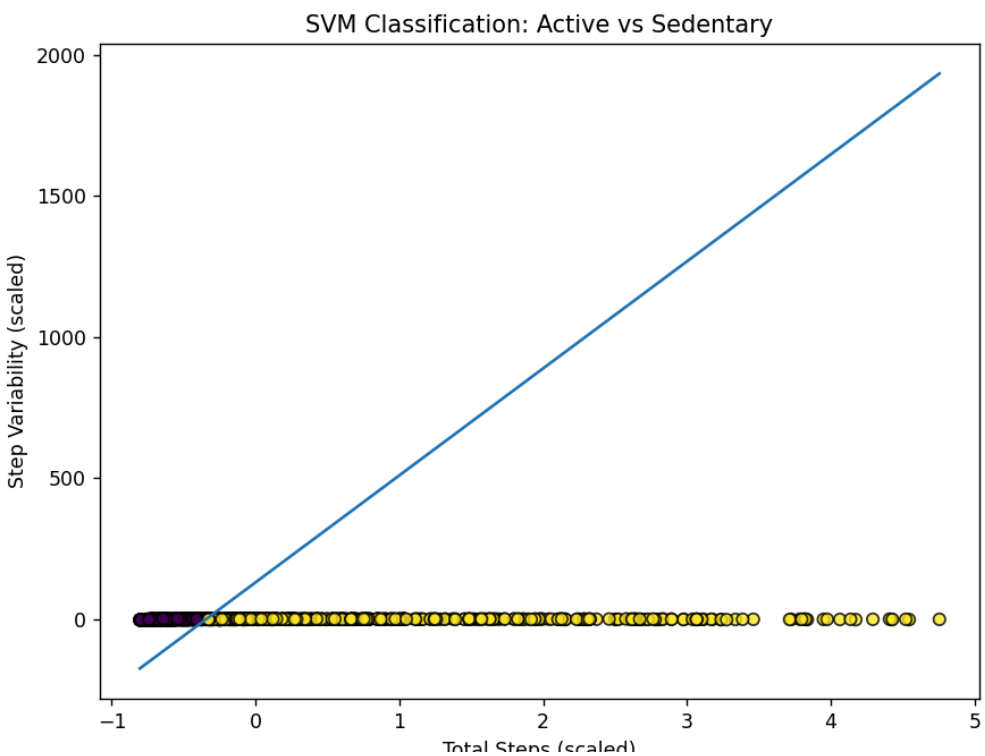
Code:

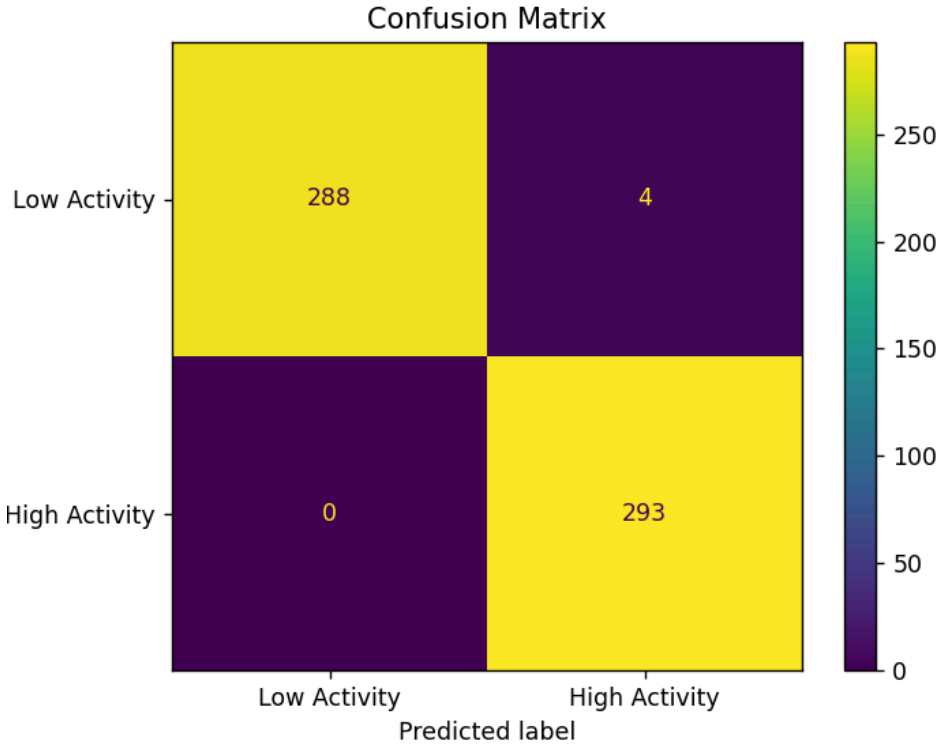
```python
import pandas as pd
import numpy as np
import glob
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, accuracy_score

files = glob.glob("*.csv")
df = pd.concat([pd.read_csv(f) for f in files], ignore_index=True)

print("Loaded rows:", df.shape[0])


df.columns = df.columns.str.lower()

df['timestamp'] = pd.to_datetime(df['timestamp'])
df['steps'] = pd.to_numeric(df['steps'])
df['hour'] = df['timestamp'].dt.floor('h')

hourly = df.groupby('hour')['steps'].agg(['sum','mean','std']).reset_index()
hourly = hourly.fillna(0)

hourly.columns = ['hour','total_steps','mean_steps','std_steps']

threshold = hourly['total_steps'].median()

hourly['label'] = np.where(hourly['total_steps'] < threshold, 0, 1)

print(hourly['label'].value_counts())

X = hourly[['total_steps','std_steps']]
y = hourly['label']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```python
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y
)

model = SVC(kernel='linear', class_weight='balanced')
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

plt.figure(figsize=(8,6))

plt.scatter(X_train[:,0], X_train[:,1], c=y_train, edgecolors='k', alpha=0.8)

plt.xlabel("Total Steps (scaled)")
plt.ylabel("Step Variability (scaled)")
plt.title("SVM Classification: Active vs Sedentary")

w = model.coef_[0]
b = model.intercept_[0]

x_vals = np.linspace(X_train[:,0].min(), X_train[:,0].max(), 100)

y_vals = -(w[0]*x_vals + b) / w[1]

plt.plot(x_vals, y_vals)

plt.show()

plt.scatter(X_train[:,0], X_train[:,1], c=y_train, edgecolors='k', alpha=0.8)

plt.xlabel("Total Steps (scaled)")
plt.ylabel("Step Variability (scaled)")
plt.title("SVM Classification: Active vs Sedentary")

plt.show()
```

```python

cm = confusion_matrix(y_test, y_pred)

plt.figure()
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Low Activity", "High Activity"]
)
disp.plot()
plt.title("Confusion Matrix")
plt.show()
```

Output:





Conclusion:

Support Vector Machine (SVM) is a powerful supervised learning algorithm used for classification tasks. It works by identifying an optimal decision boundary that maximizes the margin between different classes, which helps improve generalization to unseen data.

SVM performs well for high-dimensional and structured numerical datasets, especially when the classes are linearly separable or nearly separable. With appropriate feature scaling and proper selection of hyperparameters such as the regularization parameter and kernel type, it can produce robust and accurate models.

However, SVM can be sensitive to parameter settings and may require careful tuning for complex or non-linearly separable data. Overall, SVM is a reliable and widely used algorithm for binary classification